AI Image Generator

8. Database

Version 2.3
23 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

8. Database

8.1 Overview

File: generations.db in the project folder. Engine: SQLite 3 through Python's standard library. One table, generations, one row per image produced by the web app. An index on created_at supports newest-first listing. The schema is created on first connection; columns added in later increments are listed in NEW_COLUMNS and added with ALTER TABLE when absent. Since v2.0 rows can be amended after creation: the Upscale bar writes the upscale columns onto an existing row with db.update().

8.2 Table: generations

Identity and files

Column	Type	Meaning
id	INTEGER PK	Auto-increment row id, shown as "#N" in the UI
created_at	TEXT	ISO-8601 local timestamp, seconds precision
image_path	TEXT	Absolute path of the native PNG
file_size_kb	REAL	Size of the native PNG

Prompt

Column	Type	Meaning
user_prompt	TEXT	Exactly what was typed, including model and LoRA tags
final_prompt	TEXT	What was sent to the model after directive removal and style suffixes
negative_prompt	TEXT	Negative prompt used; empty for FLUX
parser_notes	TEXT	Parser decisions, auto-model note, LoRA notes and the sampler, joined with " - "

Settings

Column	Type	Meaning
model_id	TEXT	Model source: Hugging Face repo id or local checkpoint path
width, height	INTEGER	Native generation size
steps	INTEGER	Denoising steps actually used (after model defaults)
guidance	REAL	Guidance scale actually used
seed	INTEGER	Seed used for this image
style	TEXT	Name of the matched style preset, or NULL
batch_index, batch_size	INTEGER	Position within a multi-image request

Timing

Column	Type	Meaning
inference_seconds	REAL	Pipeline call, from start to CUDA synchronise
save_seconds	REAL	PNG encode and write of the native image
total_seconds	REAL	inference + save (excludes upscaling)
steps_per_second	REAL	steps / inference_seconds

GPU and software

Column	Type	Meaning
gpu_name	TEXT	From torch device properties
vram_total_gb	REAL	Card memory
vram_peak_gb	REAL	Peak torch allocation during this image, including an in-prompt upscale
vram_reserved_gb	REAL	Allocator reservation after the image
compute_capability	TEXT	e.g. "8.6"
driver_version	TEXT	From nvidia-smi
cuda_version, torch_version, diffusers_version	TEXT	Library versions

Telemetry (from nvidia-smi)

Column	Type	Meaning
temp_before_c, temp_after_c	REAL	GPU temperature before the pipeline call and right after it
power_before_w, power_after_w	REAL	Board power draw at the same two moments
power_limit_w	REAL	Configured power limit
gpu_util_after_pct	REAL	GPU utilisation at the after snapshot
sm_clock_mhz, mem_clock_mhz	REAL	Clock speeds at the after snapshot
fan_after_pct	REAL	Fan speed
pstate	TEXT	Performance state, P0 to P8

Upscaling (written at generation or later by the Upscale bar)

Column	Type	Meaning
upscale_target	INTEGER	Requested long side in pixels
upscaled_width, upscaled_height	INTEGER	Final size after resample and crop
upscaled_path	TEXT	Absolute path of the upscaled PNG
upscaled_size_kb	REAL	Size of that file
upscale_seconds	REAL	Time of the upscaling stage
upscale_passes	INTEGER	Number of x4 passes (1 or 2)
upscaler	TEXT	Architecture and weight file name

Video and music (v2.2)

Column	Type	Meaning
kind	TEXT	image, video or music; NULL on rows written before v2.2 means image. image_path holds the .mp4 or .wav for the other kinds
duration_s	REAL	Length of the clip or track in seconds
fps	REAL	Frames per second of a clip (24 for LTX-Video, 16 for Wan)
frames	INTEGER	Number of frames in a clip
sample_rate	INTEGER	Sample rate of a track (32000 for MusicGen, 44100 for Stable Audio)
soundtrack_path	TEXT	The _music.mp4 written when a track was laid under the clip
category	TEXT	Category chosen in the Studio or by batch.py (v2.3); the file then lives in outputs/<category>/

db.counts_by_kind() returns count, today, GPU seconds and total duration per kind; fetch_filtered() takes a kind filter; stats_by_model() counts clips and tracks like images (the Admin table calls the column uses).

Catch-all

Column	Type	Meaning
extra_json	TEXT	JSON of any record keys that have no column, for forward compatibility

Table: prompt_library (v2.3)

Every prompt generated under a category, one row per generation, so the prompts of a category can be listed, exported or reused without the images. Written by app.generate() when a category is set (the Studio box or batch.py); read by db.prompts(), db.categories() and the History category filter.

Column	Type	Meaning
id	INTEGER	Primary key
created_at	TEXT	ISO timestamp
category	TEXT	Category name as typed (case-insensitive matching)
prompt	TEXT	The prompt as written by the user (including hints such as 16:9 or seed 1001)
kind	TEXT	image (video and music do not use categories yet)
source	TEXT	studio (the Studio box and batch.py both go through the app)
generation_id	INTEGER	The generations row the prompt produced

8.3 Example queries

Average inference time by model:

```
SELECT model_id, COUNT(*) AS n, ROUND(AVG(inference_seconds), 2) AS avg_s,
       ROUND(AVG(steps_per_second), 2) AS steps_s
FROM generations GROUP BY model_id ORDER BY n DESC;
```

Hottest runs:

```
SELECT id, temp_after_c, power_after_w, user_prompt
FROM generations ORDER BY temp_after_c DESC LIMIT 5;
```

Every 4K or larger image with its file:

```
SELECT id, upscaled_width, upscaled_height, upscale_seconds, upscaled_path
FROM generations WHERE upscale_target >= 3840;
```

Estimated GPU energy of all generations, as the Admin panel computes it:

```
SELECT ROUND(SUM(inference_seconds * (power_before_w + power_after_w) / 2) / 3600, 1) AS wh
FROM generations;
```

8.4 Access from Python

```
import db
rows = db.fetch(limit=10, search="cabin")
one = db.get(4)
db.update(4, {"upscaled_width": 3840})
totals = db.stats()
```